



UNIVERSITÀ
DEGLI STUDI
FIRENZE

COMPUTER GRAPHICS AND 3D

Stefano Berretti

AU RECOGNIZER

Strumento per etichettatura automatica delle Action Units su mesh
3DMM

Paganelli Giovanni

STRUTTURA DELLA PRESENTAZIONE

1. Contesto: cos'è una Action Unit (AU)
2. Obiettivo del progetto
3. Architettura dello strumento
4. Modelli 3DMM, EMOCA e FLAME
5. AU Recognizer: flusso d'uso GUI
6. AU Recognizer: Codice interessante
7. Risultati e casi critici
8. Limiti e prospettive future

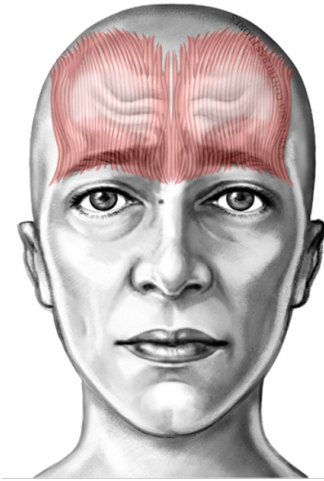
CONTESTO: COS'È UNA ACTION UNIT (AU)

- Le Action Units (AU) derivano dal sistema FACS (Facial Action Coding System - Paul Ekman e Wallace Friesen)

- Ogni AU rappresenta il movimento di specifici muscoli facciali (es. AU 1: Frontalis alza le sopracciglia)

- Le AU sono uno standard per descrivere le espressioni in modo quantitativo e oggettivo

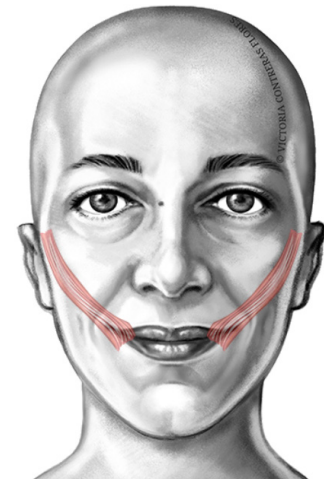
- Le AU sono classificate anche per Intensità: da A (minima) a E (massima); e Simmetria: solo lato sinistro (L), solo lato destro (R).



Artnatomy modello disegnato con Overlay muscolo Frontalis - AU 1



Artnatomy modello reale con attivazione di muscolo Frontalis - AU 1



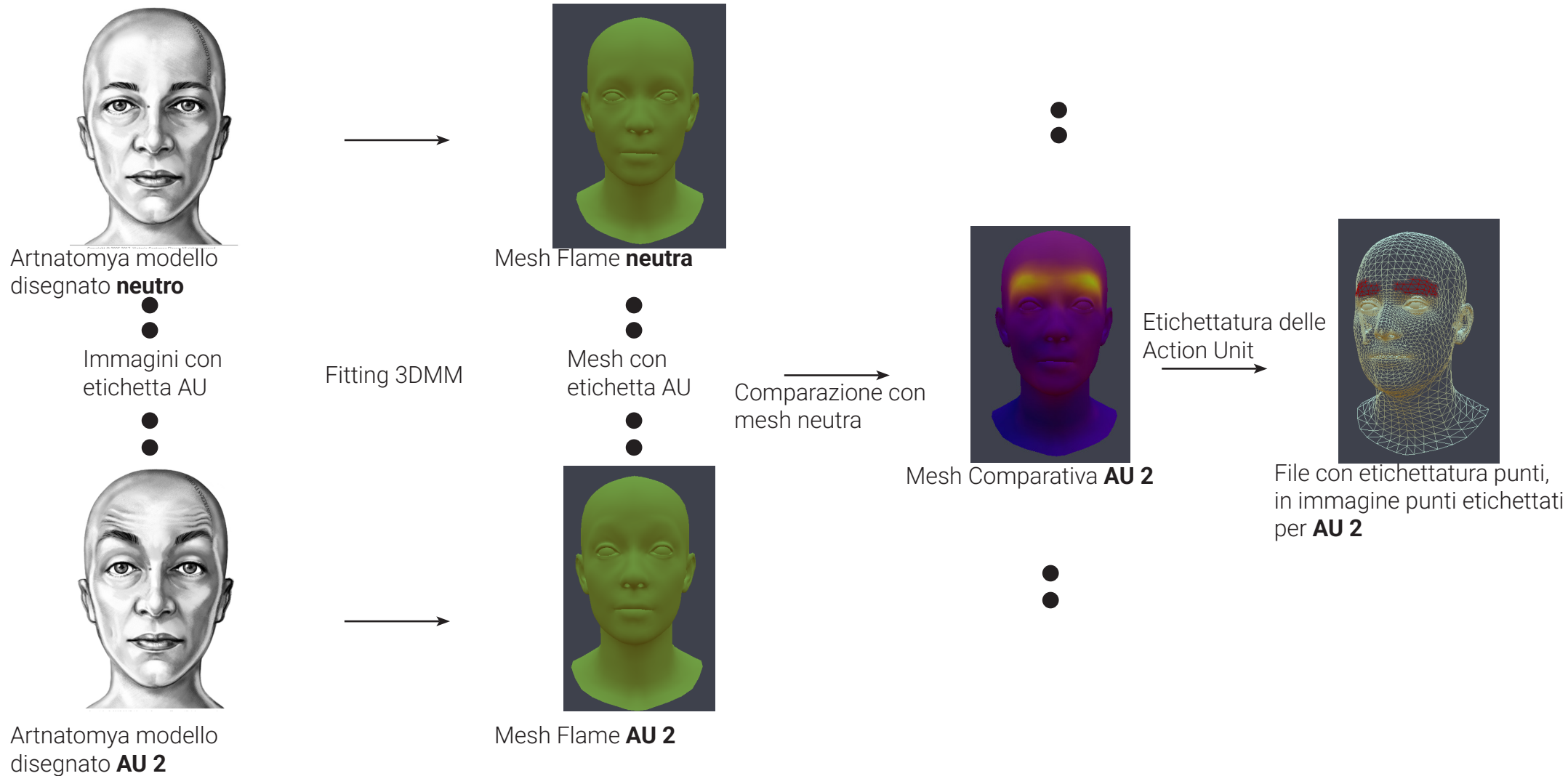
Artnatomy modello disegnato con Overlay muscolo Zigomatico - AU 12



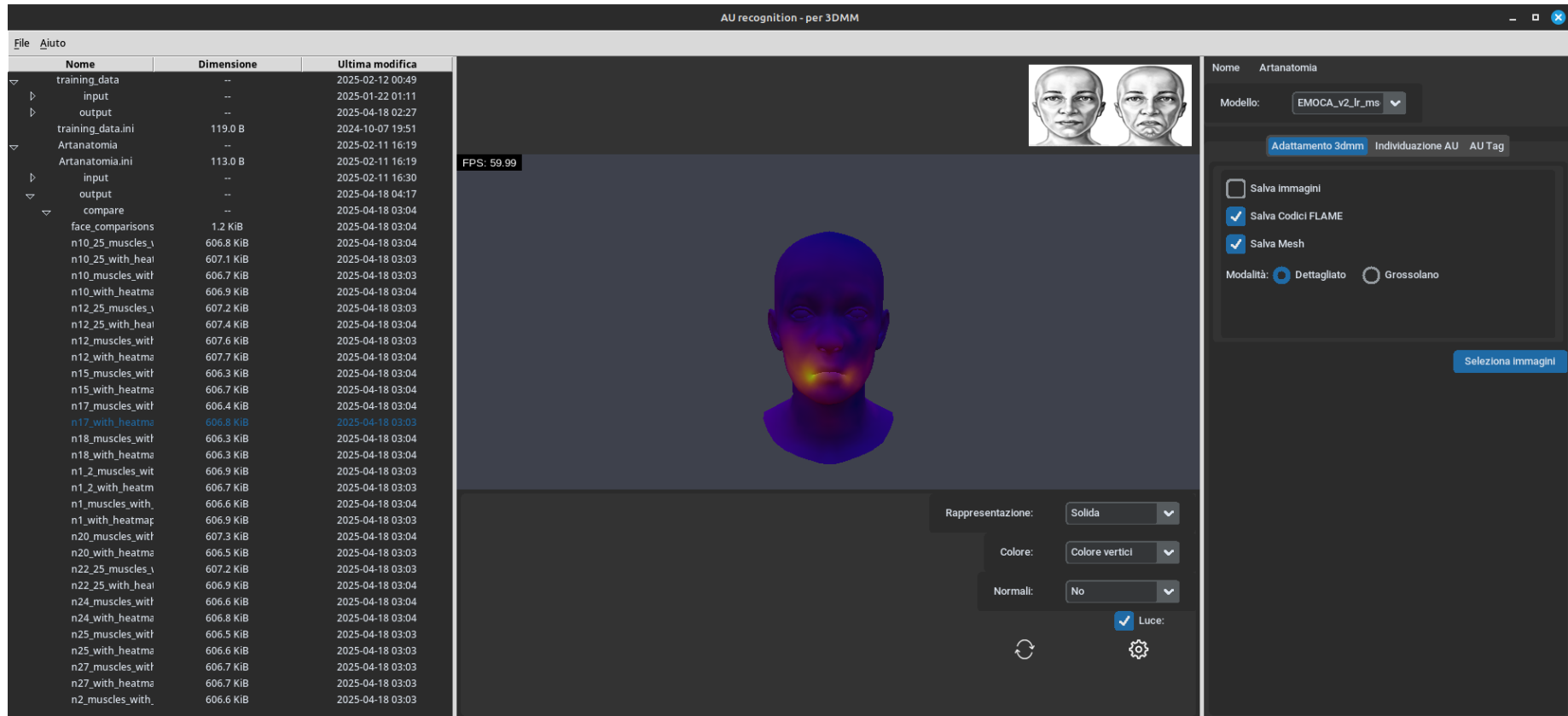
Artnatomy modello reale con attivazione di muscolo Zigomatico - AU 12

OBIETTIVO DEL PROGETTO

- Trovare una correlazione tra Action Units, muscoli attivati e punti della mesh 3D.



ARCHITETTURA DELLO STRUMENTO



- GUI realizzata in Python + Tkinter
- Fitting 3D con EMOCAv2 (topologia FLAME)
- Viewer OpenGL interattivo per esplorare le mesh
- Confronto tra mesh neutra e con AU
- Output: JSON con AU assegnate ai punti deformati

MODELLI 3DMM, EMOCA E FLAME

3D Morphable Model: è un modello statistico generativo che rappresenta oggetti tridimensionali (come volti, mani, corpi) combinando parametri numerici che controllano forma, espressione, texture e posa. Si basa su una collezione di scansioni 3D di oggetti della stessa classe.

FLAME: un 3DMM avanzato per il volto e la testa Combina forma lineare, articolazioni (mandibola, collo, occhi) e blendshapes per espressioni. È stato addestrato su oltre 3.800 individui e 33.000 scansioni 3D.

Li, T., Bolkart, T., Black, M. J., Li, H., & Romero, J. (2017). Learning a model of facial shape and expression from 4D scans. *ACM Transactions on Graphics (TOG)*, 36(6), 194:1–194:17.

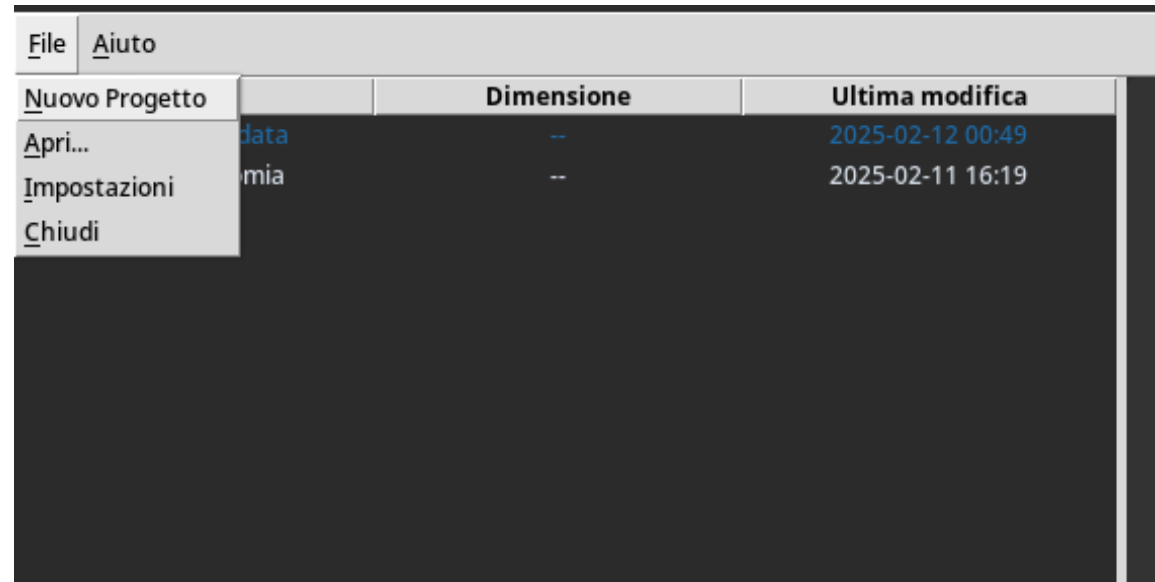
EMOCA: un framework che usa FLAME per ricostruire volti 3D da una singola immagine, enfatizzando le emozioni e preservando l'espressione originale. È stato addestrato su dataset ricchi di emozioni (es. AffectNet).

Gecer, B., Barros, P., & Wermter, S. (2021). EMOCA: Emotion Driven Monocular Face Capture and Animation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 14480–14490).

AU RECOGNIZER: FLUSSO D'USO GUI

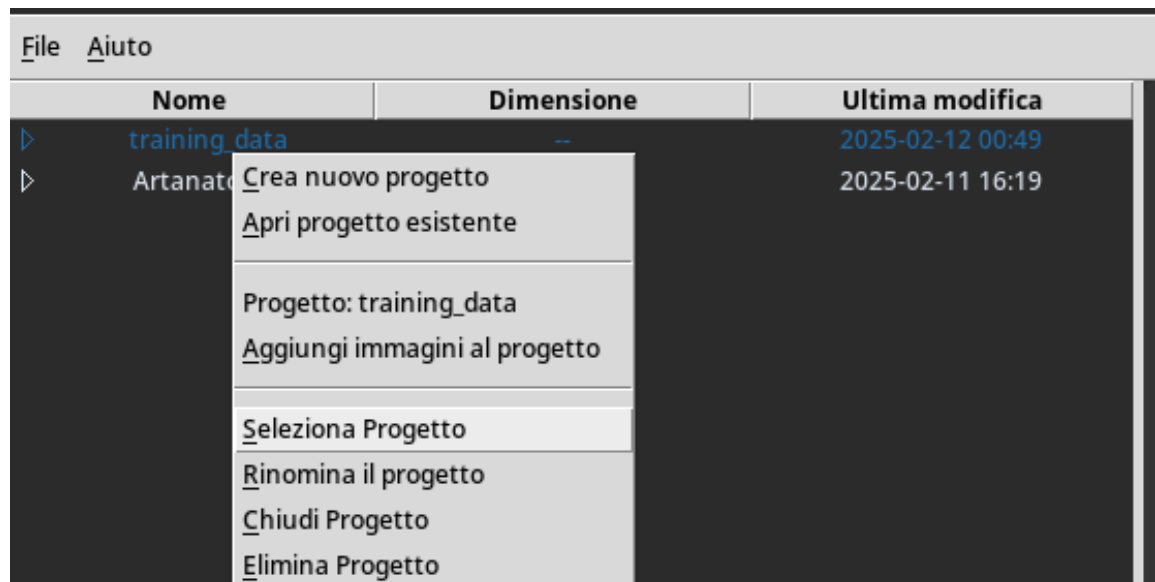
Creazione progetto e Caricamento immagini

Creazione o apertura di un progetto



Seleziona progetto e aggiungi immagini

Convenzione nome immagini soggetto_numeri_AU (es. subject1_2_5 indica AU 1, 2, e 5)



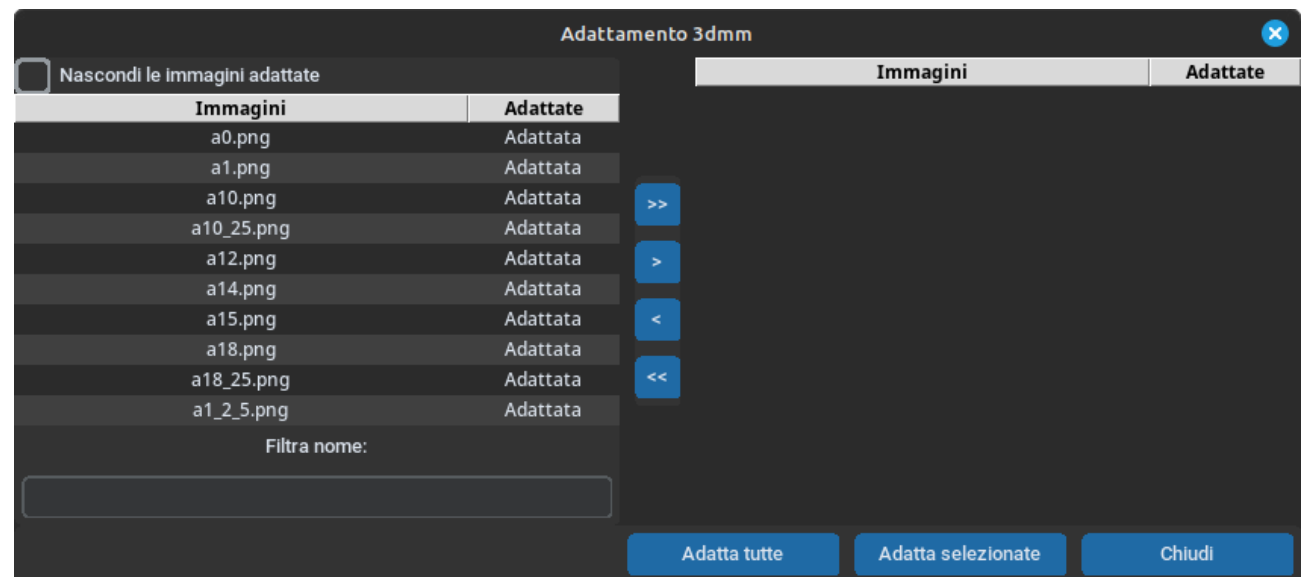
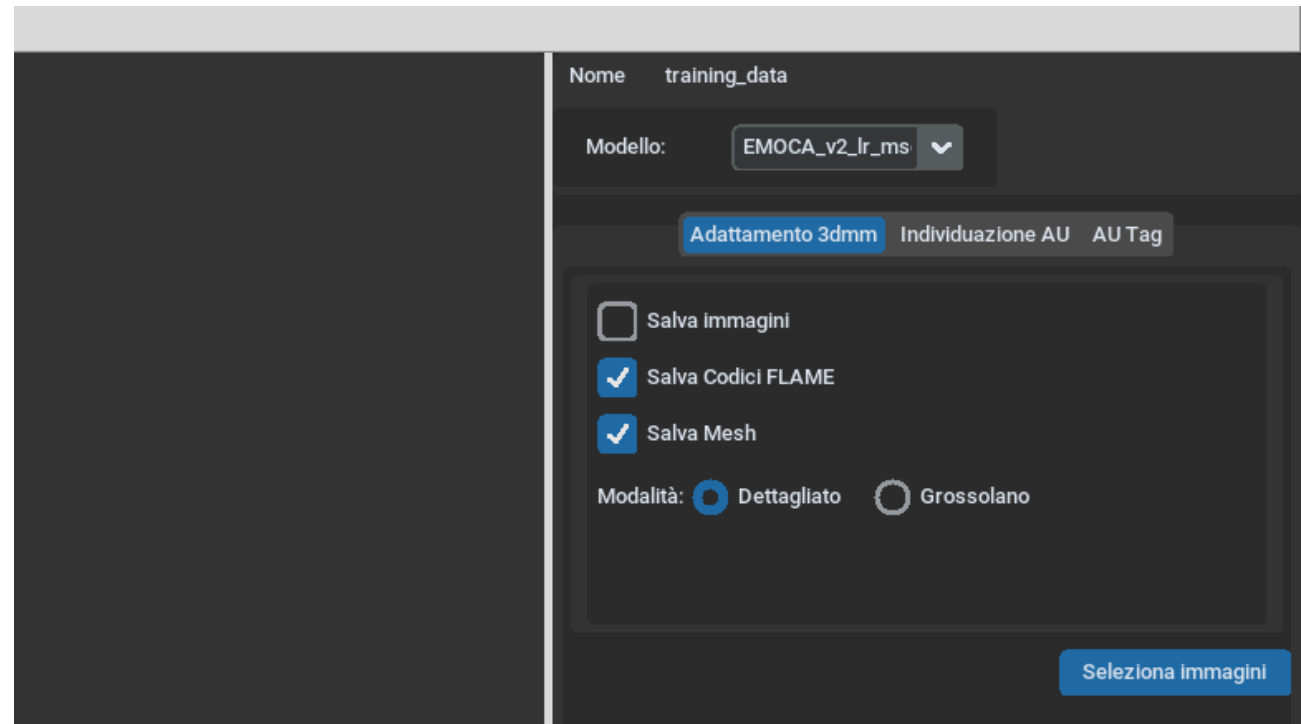
AU RECOGNIZER: FLUSSO D'USO GUI

fitting EMOCA

Fitting con EMOCA/FLAME
selezione modello e
impostazioni aggiuntive

Selezioni delle immagini da
adattare

Una volta adattate i dati si
troveranno nella cartella di
output.



AU RECOGNIZER: CODICE INTERESSANTE

fitting EMOCA

```
from abc import ABC, abstractmethod
```

```
from AU_recognizer.core.user_interface import CustomFrame
```

```
class BaseModelInterface(ABC):
```

```
    @staticmethod
```

```
    @abstractmethod
```

```
    def fit(fit_data, images_to_fit, additional_data):
        pass
```

```
    @staticmethod
```

```
    @abstractmethod
```

```
    def au_difference(mesh_neutral, mesh_list, normalization_params, project):
        """Restituisce file con differenze di punti mesh"""
        pass
```

```
    @staticmethod
```

```
    @abstractmethod
```

```
    def emoca_tag(diff_to_tag, threshold, project_data):
        """Genera file con vertici taggati"""
        pass
```

```
    @staticmethod
```

```
    @abstractmethod
```

```
    def get_ui_for_fit_data(master_widget) -> CustomFrame:
        """Restituisce la struttura widget della UI per acquisire i parametri specifici
        di fitting per il
        modello"""
        pass
```

```
class EmocaModel(BaseModelInterface):
```

```
    @staticmethod
```

```
    def fit(fit_data, images_to_fit, project_data, progress_callback=None):
```

```
        try: # Implementazione fitting con Emoca
```

```
            if isinstance(images_to_fit, list):
```

```
                images_to_fit = [str(image.resolve()) for image in images_to_fit]
```

```
            elif isinstance(images_to_fit, Path):
```

```
                images_to_fit = str(images_to_fit.resolve())
```

```
            else:
```

```
                return False
```

```
        _project_name = str(project_data.sections()[0])
```

```
        project_path = Path(project_data[_project_name][P_PATH])
```

```
        path_to_models = Path(nect_config[CONFIG][MODEL_FOLDER]) /  
        "EMOCA" / "models"
```

```
        model_name = fit_data[MF_MODEL]
```

```
        output_folder = project_path / F_OUTPUT / model_name
```

```
        mode = fit_data[MF_FIT_MODE]
```

```
        torch.cuda.empty_cache()
```

```
        emoca, conf = (load_model
```

```
                        (path_to_models, model_name, mode))
```

```
        emoca.cuda()
```

```
        emoca.eval()
```

```
        dataset = TestData(images_to_fit, max_detection=20)
```

```
        for i in auto.tqdm(range(len(dataset))):
```

```
            batch = dataset[i]
```

```
            vals, visdict = test(emoca, batch)
```

```
            current_bs = batch["image"].shape[0]
```

AU RECOGNIZER: CODICE INTERESSANTE

fitting EMOCA

```
for j in range(current_bs):
    name = batch["image_name"][j]
    sample_output_folder = Path(output_folder) / name
    sample_output_folder.mkdir(parents=True, exist_ok=True)

    if fit_data[MF_SAVE_MESH]:
        save_obj(emoca, str(sample_output_folder / "mesh_coarse.obj"),
vals, j)

    if fit_data[MF_SAVE_IMAGES]:
        save_images(output_folder, name, visdict, with_detection=True,
i=j)

    if fit_data[MF_SAVE_CODES]:
        save_codes(Path(output_folder), name, vals, i=j)
    if progress_callback:
        progress_callback((i + 1) / len(dataset))
    torch.cuda.empty_cache()
except Exception as e:
    import traceback
    traceback.print_exc()
    logger.error(f"Fitting failed: {e}")
    return False # Failure
return True
```

Inside EMOCA:

```
def test(deca, img):
    img["image"] = img["image"].cuda()
    if len(img["image"].shape) == 3:
        img["image"] = img["image"].view(1, 3, 224, 224)
    vals = deca.encode(img)
    vals, visdict = decode(deca, vals)
    return vals, visdict
```

Test(deca, img) è una funzione della libreria EMOCA e dentro vengono fatti due passaggi principali:

encode(img) Estrae codici latenti dal volto: shape, forma del viso (identità); expression, movimenti muscolari; pose, texture, lighting, camera.

decode(deca, vals) Usa i codici latenti per: Ricostruire la mesh 3D (con FLAME); Renderizzare l'immagine sintetica; Calcolare landmark 2D/3D, texture, normal map, ecc.

AU RECOGNIZER: FLUSSO D'USO GUI

Visualizzazione mesh

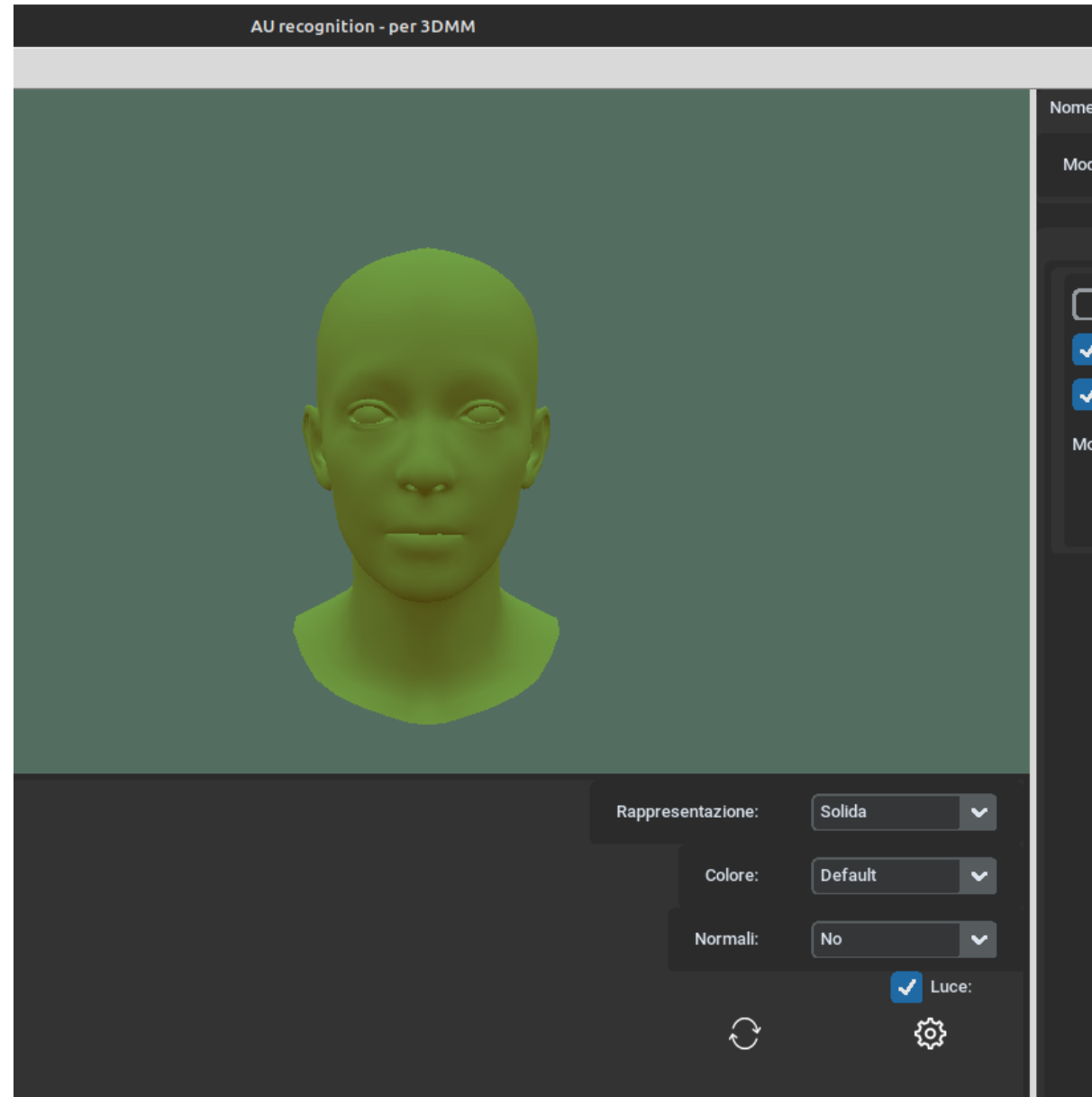
Visualizzatore OpenGL
mesh 3D, con rotazione,
traslazione, scala.

Rappresentazione: solida,
Reticolo, Puntiforme

Colore: Default, RGB della
Mesh, Tessitura

Normali: No, Si (default con
luce attiva), Mappa normale
(se presente)

Personalizzazione colori:
mesh, sfondo e luce



AU RECOGNIZER: CODICE INTERESSANTE

Visualizzazione mesh

```
def redraw(self):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glUseProgram(self.shader)
    # Set rendering mode: solid, wireframe, or points
    if self.shader_uniform['isWireframe'][GL_U_VALUE]:
        glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
    elif self.shader_uniform['isPoints'][GL_U_VALUE]:
        glPolygonMode(GL_FRONT_AND_BACK, GL_POINT)
    else:
        glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)
    # Set up the projection matrix (perspective projection)
    aspect_ratio = self._canvas_w / self._canvas_h
    proj = perspective(45.0, aspect_ratio, 0.1, 100.0)
    # View matrix (camera)
    view = look_at(eye=self.camera_position, target=np.array([0.0, 0.0, 0.0]), up=np.array([0.0, 1.0, 0.0]))
    # Model matrix (rotation)
    model = np.eye(4)
    rot_matrix = quaternion_to_matrix(self._rotation)
    model[:3, :3] = rot_matrix[:3, :3]
    model = np.dot(np.array([
        [1, 0, 0, self.panning[0]],
        [0, 1, 0, self.panning[1]],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ]), model)
    # Apply scaling last
    scale_matrix = np.eye(4)
    scale_matrix[0, 0] = self.scale # Uniform scale factor for x-axis
    scale_matrix[1, 1] = self.scale # Uniform scale factor for y-axis
    scale_matrix[2, 2] = self.scale # Uniform scale factor for z-axis
    # Apply scaling to the model
    model = np.dot(scale_matrix, model)
```

```
self.update_shader_uniforms([("view", view.T),
                              ("proj", proj.T),
                              ("model", model.T)])

for material_name, vao in self.vertex_array_object.items():
    # Bind the VAO
    glBindVertexArray(vao)
    # Bind the appropriate texture for the material
    if material_name in self.normal_maps and self.shader_uniform["useNormalMap"][GL_U_VALUE] is True:
        glActiveTexture(GL_TEXTURE1) # Set to use texture unit 1 for the normal
        map
        glBindTexture(GL_TEXTURE_2D, self.normal_maps[material_name])

    # Bind the appropriate texture for the material
    if material_name in self.textures and self.shader_uniform["useTexture"][GL_U_VALUE] is True:
        glActiveTexture(GL_TEXTURE0) # Set to use texture unit 0 for the diffuse
        texture
        glBindTexture(GL_TEXTURE_2D, self.textures[material_name])

    if ('index_data' in self.model['batched_data'][material_name]
        and self.model['batched_data'][material_name]['index_data'] is not None):
        glDrawElements(GL_TRIANGLES, len(self.model['batched_data'][material_name]['index_data']),
                        GL_UNSIGNED_INT,
                        None)
    else:
        glDrawArrays(GL_POINTS, 0, len(self.model['batched_data'][material_name]['vertex_data']))
        glBindTexture(GL_TEXTURE_2D, 0)
        glBindVertexArray(0) # Unbind the VAO when done
        glUseProgram(0)
```

AU RECOGNIZER: CODICE INTERESSANTE

Shader

The vertex shader code

```
vertex_shader_code = ""
```

```
#version 330 core
```

```
layout(location = 0) in vec3 inPosition;
```

```
layout(location = 1) in vec3 inVertexColor;
```

```
layout(location = 2) in vec2 inTexCoords;
```

```
layout(location = 3) in vec3 aNormal; // Geometric normal
```

```
layout(location = 4) in vec3 aTangent; // Tangent vector
```

```
uniform mat4 model;
```

```
uniform mat4 view;
```

```
uniform mat4 proj;
```

```
out vec2 fragTexCoords;
```

```
out vec3 fragColor;
```

```
out mat3 TBN;
```

```
out vec3 fragNormal;
```

```
void main() {
```

```
    gl_Position = proj * view * model * vec4(inPosition, 1.0);
```

```
    fragTexCoords = inTexCoords;
```

```
    fragColor = inVertexColor;
```

```
    fragNormal = normalize(mat3(transpose(inverse(model))) * aNormal);
```

```
    vec3 T = normalize(vec3(model * vec4(aTangent, 0.0)));
```

```
    vec3 N = normalize(vec3(model * vec4(aNormal, 0.0)));
```

```
    // re-orthogonalize T with respect to N
```

```
    T = normalize(T - dot(T, N) * N);
```

```
    // then retrieve perpendicular vector B with the cross product of T and N
```

```
    vec3 B = cross(N, T);
```

```
    TBN = mat3(T, B, N);
```

```
}
```

```
""
```

Fragment Shader for Solid Color / Per-Vertex Color

```
fragment_shader_code = ""
```

```
#version 330 core
```

```
in vec2 fragTexCoords;
```

```
in vec3 fragColor;
```

```
in mat3 TBN;
```

```
in vec3 fragNormal;
```

```
out vec4 FragColor;
```

```
uniform bool useTexture; // Toggle texture usage
```

```
uniform bool useNormalMap; // Toggle normal mapping
```

```
uniform sampler2D textureMap; // Diffuse texture
```

```
uniform sampler2D normalMap; // Normal map
```

```
uniform vec3 solidColor; // Default solid color
```

```
uniform vec3 wireframeColor; // Wireframe color
```

```
uniform vec3 pointsColor; // Points color
```

```
uniform bool useLight; // Toggle normal mapping
```

```
uniform vec3 skyColor; // Light color from above
```

```
uniform vec3 groundColor; // Light color from below
```

AU RECOGNIZER: CODICE INTERESSANTE

Shader

```
uniform bool isWireframe;
uniform bool isPoints;
uniform bool useVertexColor;
uniform bool useNormal;    // Toggle normal mapping

vec3 applyLighting(vec3 color, vec3 normal) {
    float blendFactor = normal.y * 0.5 + 0.5;
    // Mix between sky color and ground color
    vec3 hemisphericLight = mix(groundColor, skyColor, blendFactor);
    return color * hemisphericLight;
}

vec3 calculateColor() {
    if (isWireframe) {
        return useVertexColor ? fragColor : wireframeColor;
    } else if (isPoints) {
        return useVertexColor ? fragColor : pointsColor;
    } else if (useTexture) {
        return texture(textureMap, fragTexCoords).rgb;
    } else {
        return useVertexColor ? fragColor : solidColor;
    }
}

void main() {
    vec3 finalColor = calculateColor();
    vec3 normal = fragNormal;
    if (useNormalMap) {
        // Apply normal mapping if enabled
        normal = texture(normalMap, fragTexCoords).rgb * 2.0 - 1.0;
        // Tangent space transformation
        normal = normalize(TBN * normal); // Transform to world space
    }
    if (useLight) {
        finalColor = applyLighting(finalColor, normal);
    } else if (useNormal) {
        vec3 normalEffect = 0.5 + 0.5 * normal; // Map normal from [-1,1] to [0,1]
        finalColor *= normalEffect;           // Blend with base color
    }
    FragColor = vec4(finalColor, 1.0);
}
```

AU RECOGNIZER: FLUSSO D'USO GUI

Individuazione Action Unit

Individuazione Action Unit

Seleziona Mesh volto Neutro

Seleziona Mesh con volti da
comparare

Opzioni aggiuntive di
normalizzazione posa e
identità (relativi ai parametri
delle 3DMM, vedremo nel
codice)

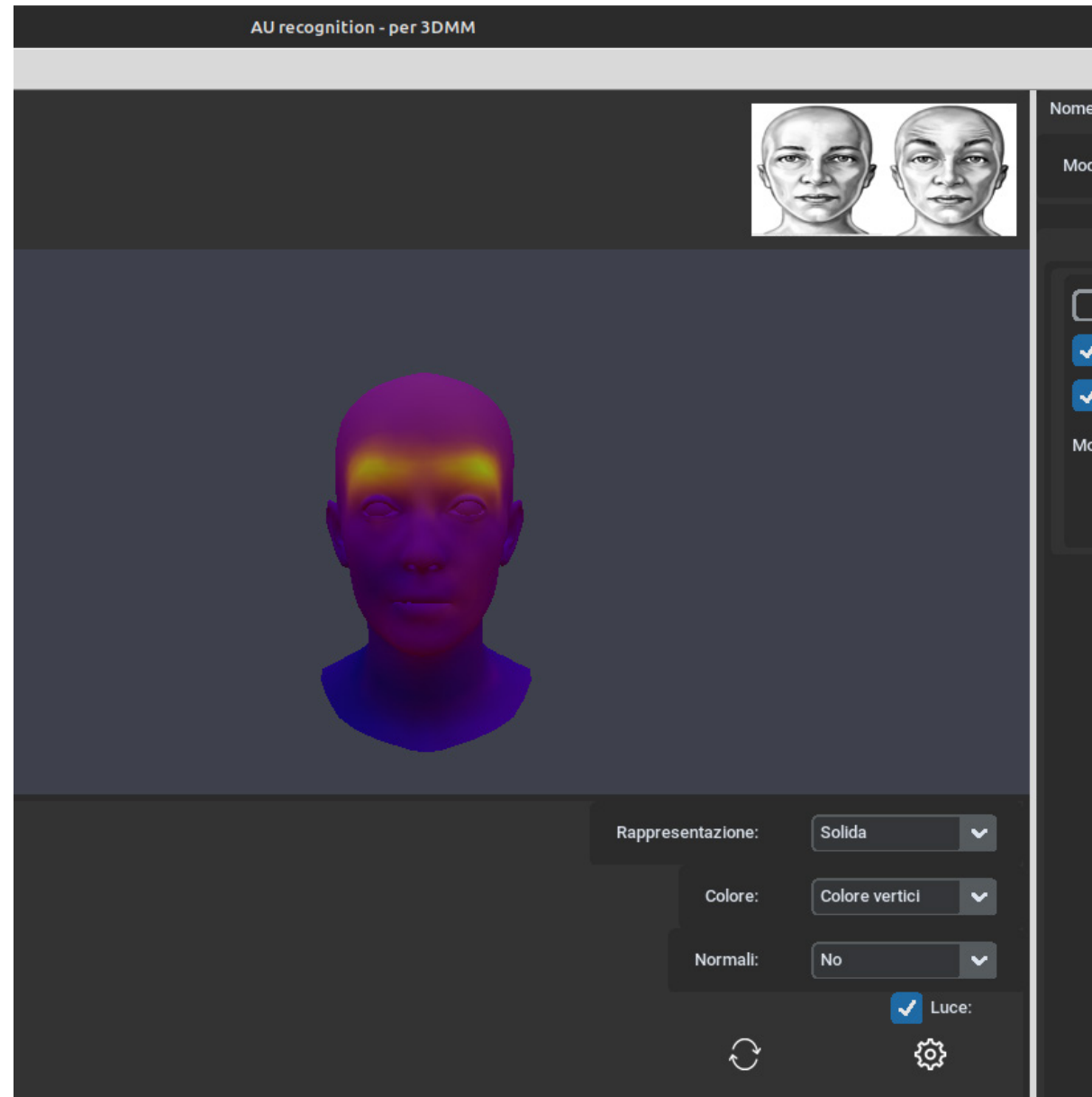
The screenshot displays the 'AU RECOGNIZER' GUI with the 'Individuazione AU' tab selected. The interface includes a 'Nome' field with the value 'training_data' and a 'Modello:' dropdown menu set to 'EMOCA_v2_lr_ms'. Below these are three tabs: 'Adattamento 3dmm', 'Individuazione AU' (active), and 'AU Tag'. The 'Individuazione AU' tab contains two sections: 'mesh volto neutrale' and 'mesh volti comparare'. Each section lists ten mesh identifiers: s20B_25A00, w4B00, s12B_23D00, pg4C_5D_7A_23A00, pb4C_5C_17D_24E_38E00, n100, pf1C_2C_5E_12AL00, ph4B_7B_10C_17D00, s15D_23D_38B00, and s6B_7B_12B_15B00. At the bottom, there are two checkboxes: 'Normalizza Posa' (checked) and 'Normalizza Identità' (unchecked). A 'Cerca e carica dati' button is visible at the bottom right.

Nome	training_data	
Modello:	EMOCA_v2_lr_ms	
Adattamento 3dmm	Individuazione AU	AU Tag
mesh volto neutrale		
s20B_25A00		
w4B00		
s12B_23D00		
pg4C_5D_7A_23A00		
pb4C_5C_17D_24E_38E00		
n100		
pf1C_2C_5E_12AL00		
ph4B_7B_10C_17D00		
s15D_23D_38B00		
s6B_7B_12B_15B00		
mesh volti comparare		
s20B_25A00		
w4B00		
s12B_23D00		
pg4C_5D_7A_23A00		
pb4C_5C_17D_24E_38E00		
n100		
pf1C_2C_5E_12AL00		
ph4B_7B_10C_17D00		
s15D_23D_38B00		
s6B_7B_12B_15B00		
☒ Normalizza Posa ☐ Normalizza Identità		
Cerca e carica dati		

AU RECOGNIZER: FLUSSO D'USO GUI

Output Comparazioni

Mesh con heatmap delle differenze tra le mesh dei due volti. Visualizzabile nel viewer OpenGL



AU RECOGNIZER: CODICE INTERESSANTE

Output Comparazioni

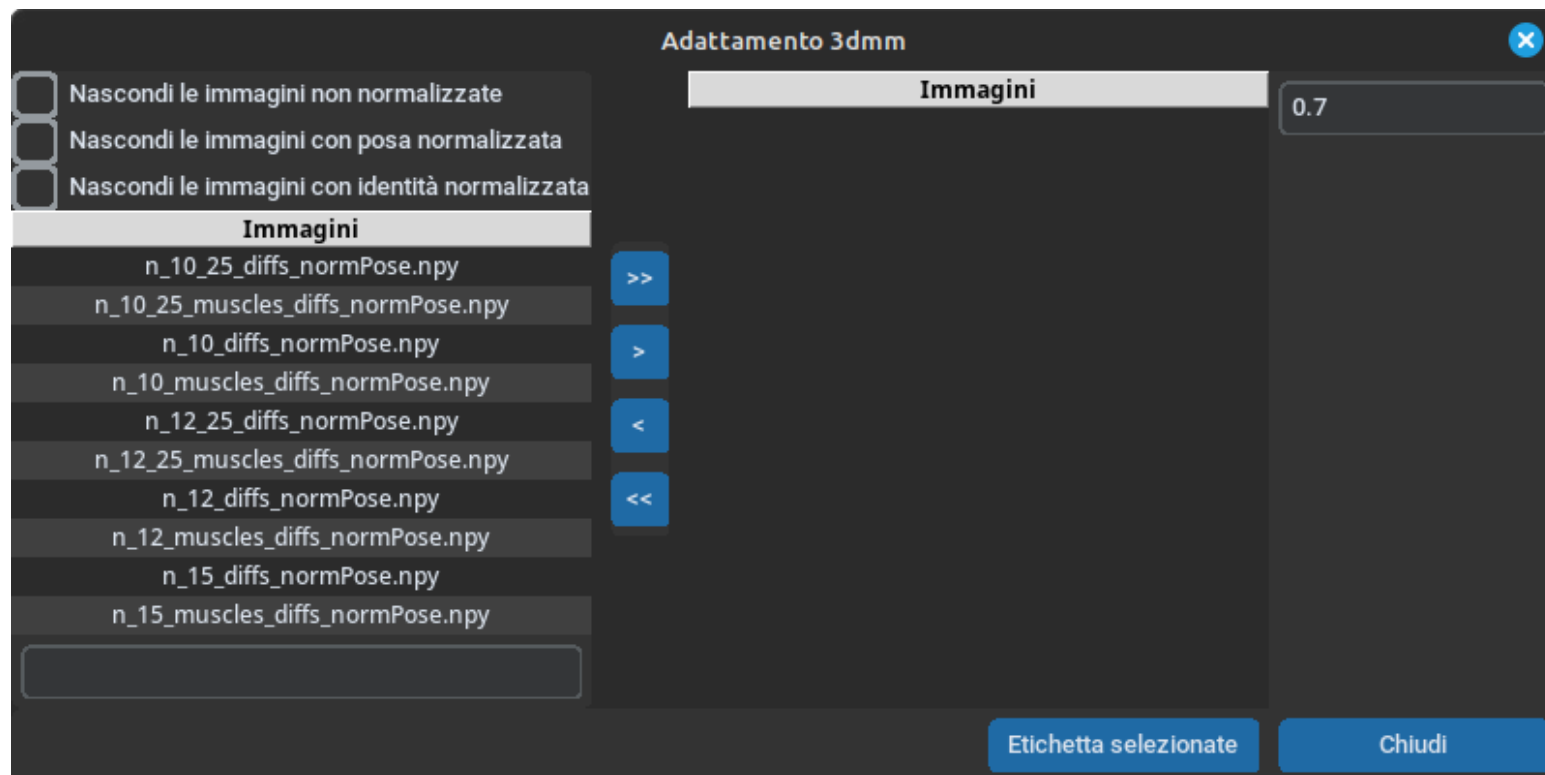
```
class EmocaModel(BaseModelInterface):
    @staticmethod
    def au_difference(mesh_neutral, mesh_list, normalization_params, project):
        # Calcolo differenze
        _project_name = str(project.sections()[0])
        output_path = Path(project[_project_name][P_PATH]) / F_OUTPUT / F_COMPARE
        output_path.mkdir(parents=True, exist_ok=True)
        neutral_face_path = Path(mesh_neutral)
        neutral_obj = OBJ(filepath=neutral_face_path / "mesh_coarse.obj", generate_normals=False)
        n_verts = neutral_obj.get_vertices()
        neutral_pose = np.load(neutral_face_path / 'pose.npy')
        neutral_shape = np.load(neutral_face_path / 'shape.npy')
        # load emoca model
        path_to_models = Path(nect_config[CONFIG][MODEL_FOLDER]) / "EMOCA" / "models"
        model_name = "EMOCA_v2_lr_mse_20"
        mode = "detail"
        # Build a suffix string based on normalization options
        suffix = ""
        if normalization_params["pose"]:
            suffix += "_normPose"
        if normalization_params["identity"]:
            suffix += "_normIdentity"
        # 0) clear memory
        torch.cuda.empty_cache() # Clear any cached GPU memory
        # 1) Load the model
        emoca, conf = load_model(path_to_models, model_name, mode)
        emoca.cuda()
        emoca.eval()
        for face_path in mesh_list:
            face_path = Path(face_path)
            face_obj = OBJ(filepath=face_path / "mesh_coarse.obj", generate_normals=False)
            f_exp = np.load(face_path / 'exp.npy')
            f_pose = np.load(face_path / 'pose.npy')
            f_shape = np.load(face_path / 'shape.npy')
            shape = torch.from_numpy(neutral_shape if normalization_params["pose"] else f_shape)
            exp = torch.from_numpy(f_exp)
```

```
            pose = torch.from_numpy(neutral_pose if normalization_params["identity"] else f_pose)
            shape = shape.unsqueeze(0) # Shape: [1, 100]
            exp = exp.unsqueeze(0) # Shape: [1, 50]
            pose = pose.unsqueeze(0) # Shape: [1, 6]
            device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
            shape = shape.to(device)
            exp = exp.to(device)
            pose = pose.to(device)
            if not isinstance(emoca.deca.flame, FLAME_mediapipe):
                verts, landmarks2d, landmarks3d = emoca.deca.flame(shape_params=shape,
                    expression_params=exp,
                    pose_params=pose)
            else:
                verts, landmarks2d, landmarks3d, landmarks2d_mediapipe = emoca.deca.flame(shape_params=shape,
                    expression_params=exp,
                    pose_params=pose)
            face_verts = verts.squeeze(0) # Remove the batch dimension
            face_verts_np = face_verts.cpu().numpy() # Convert PyTorch tensor to NumPy array
            diffs = np.linalg.norm(face_verts_np - n_verts, axis=1) # Shape: [5023]
            normalized_diffs = (diffs - np.min(diffs)) / (np.max(diffs) - np.min(diffs))
            colormap = cm.get_cmap("plasma") # Choose a colormap
            vertex_colors = colormap(normalized_diffs)[:,:3] # RGB values for each vertex
            compare_obj = copy.deepcopy(face_obj)
            compare_obj.set_vertex_colors(vertex_colors)
            compare_obj.save(output_path / f"{face_path.name[:-2]}_with_heatmap{suffix}.obj")
            # save diff
            # Usa una regex per inserire un "_" tra lettere e numeri
            modified_name = re.sub(r'([A-Za-z]+)(\d)', r'\1_\2', face_path.name[:-2])
            np.save(output_path / f"{modified_name}_diffs{suffix}.npy", diffs) # Save as binary
        return True
```

AU RECOGNIZER: FLUSSO D'USO GUI

Tag delle Action Unit

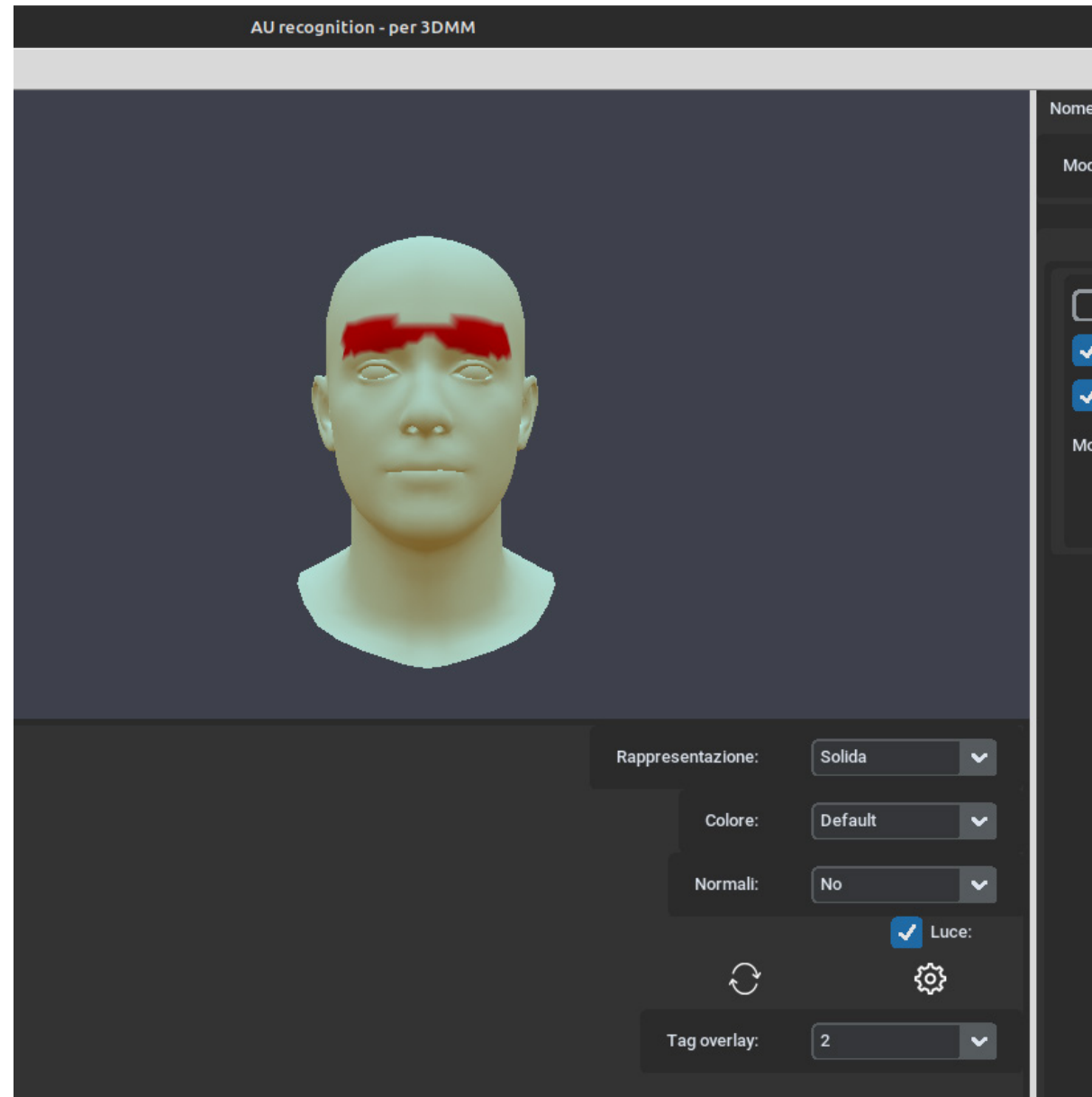
Finestra per la selezione delle immagini da prendere in considerazione per il tagging con valore di threshold inserito manualmente.



AU RECOGNIZER: FLUSSO D'USO GUI

Output Tag

Selezionando il file .JSON di output verrà mostrata una mesh neutra con una casella per selezionare l'AU che mostrerà un overlay dei punti attivati.



AU RECOGNIZER: CODICE INTERESSANTE

Tag delle Action Unit

```
@staticmethod
def emoca_tag(diff_to_tag, threshold, project_data):
    logger.debug("Tagging vertices based on threshold...")
    project_name = str(project_data.sections()[0])
    output_path = Path(project_data[project_name][P_PATH]) / F_OUTPUT
    / F_TAG
    output_path.mkdir(parents=True, exist_ok=True) # Ensure the directory
    exists

    tagged_vertices = [[] for _ in range(5023)] # List of sequences for each
    vertex

    for diff_file in diff_to_tag:
        if diff_file.suffix.lower() == '.numpy':
            code = extract_codes(diff_file.stem) # Extract only the codes
            if not code:
                logger.warning(f"no code in file name {diff_file.stem}")
                continue # Skip files with no valid code
            diffs = np.load(diff_file) # Load the NumPy array
            if diffs.shape[0] != 5023:
                logger.warning(f"File {diff_file} has {diffs.shape[0]} vertices instead
                of 5023. Skipping.")
                continue # Skip files with unexpected shapes

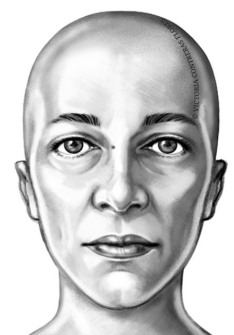
            # Normalize safely (avoid division by zero)
            min_val, max_val = np.min(diffs), np.max(diffs)
            if max_val == min_val:
                normalized_diffs = np.zeros_like(diffs) # If all values are the same,
                set them to 0
            else:
                normalized_diffs = (diffs - min_val) / (max_val - min_val)

    for i, norm_diff in enumerate(normalized_diffs):
        if norm_diff >= threshold:
            tagged_vertices[i].append(code) # Append the tag code

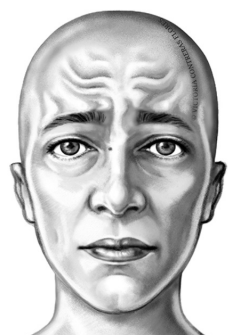
    # Save tagged data to a file
    output_file = output_path / f"tagged_vertices_{threshold}.json"
    with open(output_file, "w") as f:
        json.dump(tagged_vertices, f, indent=2) # Pretty-print for readability
    logger.info(f"Tagging completed. Results saved to {output_path}")
    return True
```

RISULTATI E CASI CRITICI

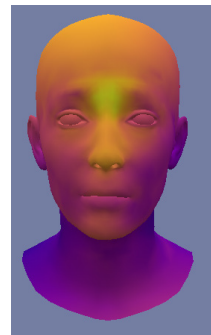
Confronto manuale tra risultato e valori attesi



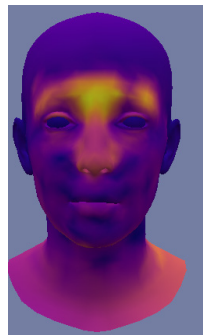
Modello **neutro**



Modello **AU 1**



Diff **AU 1**



Diff **AU 1** NormIdentity



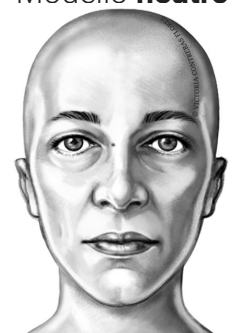
Tag **AU 1** soglia 0.3



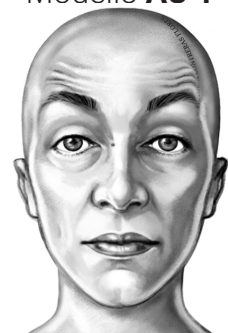
Tag **AU 1** soglia 0.5



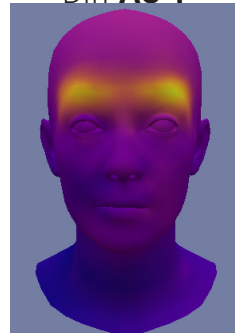
Tag **AU 1** soglia 0.7



Modello **neutro**



Modello **AU 2**



Diff **AU 2**



Diff **AU 2** NormIdentity



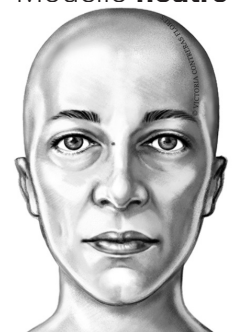
Tag **AU 2** soglia 0.3



Tag **AU 2** soglia 0.5



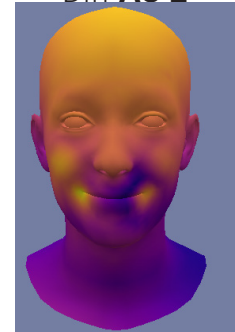
Tag **AU 2** soglia 0.7



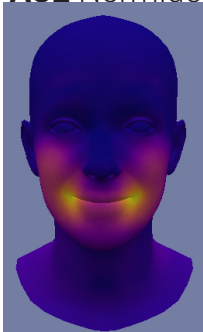
Modello **neutro**



Modello **AU 12**



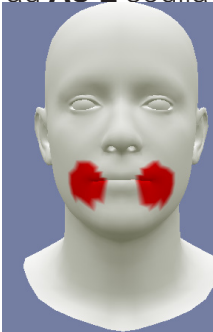
Diff **AU 12**



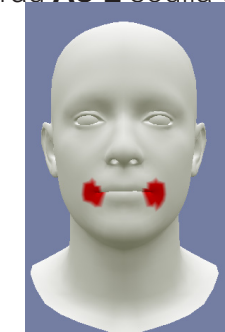
Diff **AU 12** NormIdentity



Tag **AU 12** soglia 0.3



Tag **AU 12** soglia 0.5



Tag **AU 12** soglia 0.7

RISULTATI E CASI CRITICI

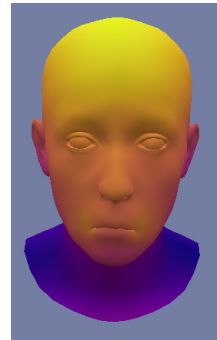
Confronto manuale tra risultato e valori attesi



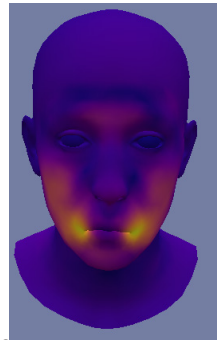
Modello **neutro**



Modello **AU 1**



Diff **AU 1**



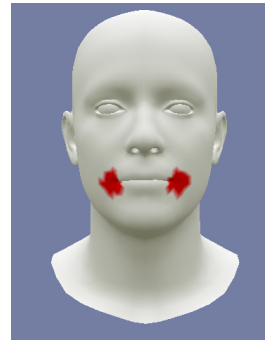
Diff **AU 1** NormIdentity



Tag **AU 1** soglia 0.3



Tag **AU 1** soglia 0.5



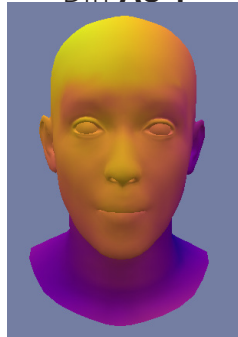
Tag **AU 1** soglia 0.7



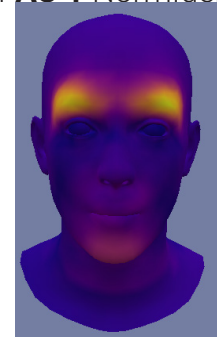
Modello **neutro**



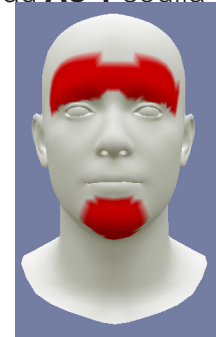
Modello **AU 2**



Diff **AU 2**



Diff **AU 2** NormIdentity



Tag **AU 2** soglia 0.3



Tag **AU 2** soglia 0.5



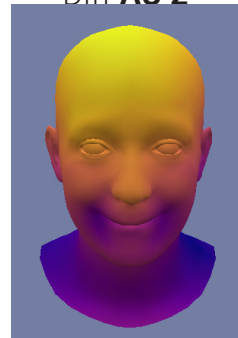
Tag **AU 2** soglia 0.7



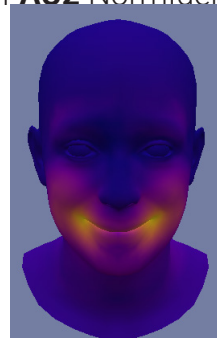
Modello **neutro**



Modello **AU 12**



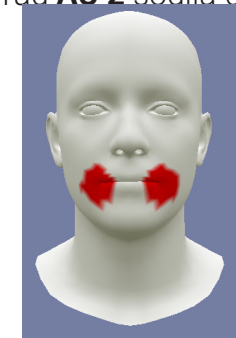
Diff **AU 12**



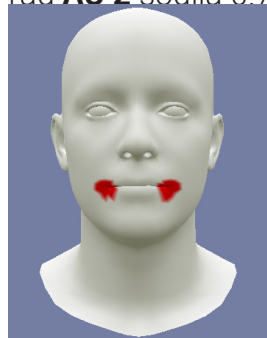
Diff **AU 12** NormIdentity



Tag **AU 12** soglia 0.3



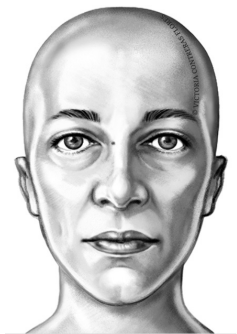
Tag **AU 12** soglia 0.5



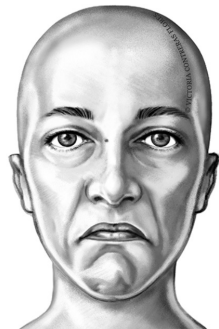
Tag **AU 12** soglia 0.7

RISULTATI E CASI CRITICI

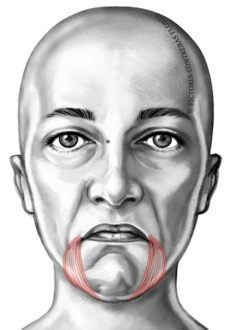
Confronto manuale tra risultato e valori attesi



Modello **neutro**



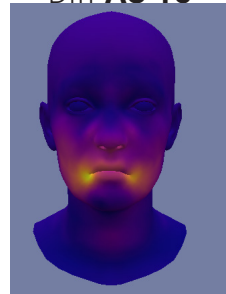
Modello **AU 15**



Muscolo **AU 15**



Diff **AU 15**



Diff **AU 15** NormId



Tag **AU 15** soglia 0.3



Tag **AU 15** soglia 0.3



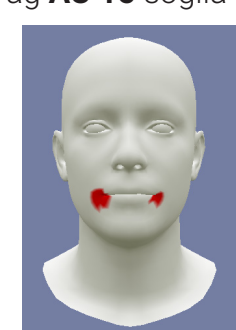
Tag **AU 15** soglia 0.5



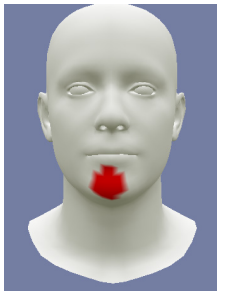
Tag **AU 15** soglia 0.5



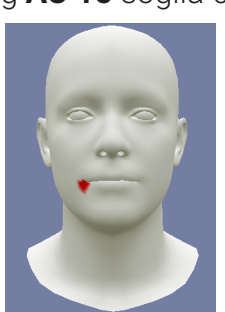
Tag **AU 15** soglia 0.7



Tag **AU 15** soglia 0.7



Tag **AU 15** soglia 0.9



Tag **AU 15** soglia 0.9

LIMITI ATTUALI E SVILUPPI FUTURI

Limite attuale	Possibile evoluzione
Supporto per AU multiple: Assegno tutte le AU ai punti mossi (es. AU 1_2_5)	Assegnazione più precisa AU, basandosi su regioni della mesh, o considerare solo casi con AU singole
Supporto per intensità AU o asimmetria: Assegno tutte le AU ai punti mossi (es. AU 1E_2B_5C)	Riconoscere e gestire l'intensità e l'asimmetria, sfruttando l'intensità della differenza punto-punto.
Soglia unica e statica per tutto il volto per il tag	Soglia dinamica o adattiva in base alla regione facciale
Il dataset usato era molto piccolo e ridotto, con un numero limitato di soggetti e AU annotate.	Utilizzare altri dataset (es. D3DFACS), che includano annotazioni di Action Units.

CONCLUSIONE

uno strumento GUI per il tagging semi-automatico delle Action Units su mesh 3D, integrando la 3DMM FLAME e EMOCA.

Permette di analizzare le deformazioni facciali e il tagging tramite viewer in OpenGL

Grazie per l'attenzione.